

Iterative Adversarial Audit as Experimental Design:

How Six Rounds of Structured Criticism Shaped a Deception Research Program

Thomas Edrington*

Liberation Labs / Transparent Humboldt Coalition

July 2026

AI Disclosure: AI contributors to this work include CC (Coalition Code) and Kavi, implemented on the Claude architecture (Anthropic). Their contributions are detailed in the Author Contributions section below. T. Edrington accepts accountability as corresponding human author.

Abstract

We describe a methodology for AI safety research in which every experiment is preceded and followed by a structured adversarial audit whose explicit goal is to identify confounds that explain the result without invoking the claimed mechanism. When a confound is found, it becomes the next experiment. We present this as a case study from a deception research program where six rounds of audit transformed an initial “deception direction” claim—which would have been published and was wrong—into a decomposition of the direction into consequentiality and deception-specific components, and ultimately into a targeted behavioral correction with placebo controls. We document every claim that was killed, why it was killed, what experiment replaced it, and what survived. The progression illustrates a general principle: adversarial review does not merely validate findings—it generates the experimental sequence that discovers them.

*Infrastructure, adversarial audit, editorial. Correspondence: info@digitaldisconnections.com

Contents

1	Introduction	3
1.1	The Protocol	3
2	The Six Rounds	4
2.1	Round 1: “We Found a Deception Direction” → KILLED (Content Confound) . . .	4
2.2	Round 2: “The Direction Transfers Across Scenarios” → PARTIAL (Structural Confound)	4
2.3	Round 3: “The Direction Works for Non-Threat Deception” → PARTIAL (Consequentiality Confound)	4
2.4	Round 4: “Consequentiality Is the Base Signal, Deception Is the Amplifier” → PASSED (with residualization caveats)	5
2.5	Round 5: “The Deception Component Survives Orthogonalization” → KILLED then PASSED (Circularity Regression)	5
2.6	Round 6: “The Loop Corrects Deception” → SPLIT VERDICT	6
3	What the Audits Killed: A Complete Registry	7
4	What the Audits Taught	7
4.1	Methodology Can Regress	7
4.2	The Most Publishable Number Is Often the Least Defensible	8
4.3	Confounds Are Generative	8
4.4	Exploratory Analysis Without Significance Testing Is Dangerous	8
4.5	The Audit Protocol Must Be Preserved Verbatim	8
5	Recommendations for Adoption	8
6	Conclusion	9
A	Complete Audit Reports (Six Rounds, Verbatim)	10
B	Killed-Claims Registry with Full Evidence Chains	11
C	Code for the Audit Protocol Automation	11

1 Introduction

The standard workflow in mechanistic interpretability research is: extract a feature, validate it on a held-out set, publish. Adversarial review, when it happens, occurs after publication—in peer review, on social media, or through replication attempts. By then, the original claim has shaped the field’s priors and downstream work.

We propose an alternative: make adversarial review the engine of the research program, not its endpoint. Each experiment is designed to answer the previous audit’s strongest objection. The result is not a single finding defended against criticism, but a sequence of progressively stronger findings that could not have been designed without the criticism.

This paper documents six rounds of this process applied to a deception research program on a 27B-parameter language model. The initial finding—a “deception direction” with apparently enormous effect sizes—would have been published as a clean positive result. It was wrong. The direction primarily encoded output-consequentiality, not deception. We would not have discovered this without the audit that demanded the consequentiality control. And we would not have discovered the targeted correction without the decomposition that the consequentiality finding produced.

Every killed claim is documented. Every audit report is provided in full as supplementary material. The goal is not to advocate for a specific finding but to demonstrate that the audit process itself is a research methodology worth adopting.

1.1 The Protocol

Each stage follows the same structure:

1. **Design:** experiment addresses the previous audit’s primary objection
2. **Pre-flight audit:** adversarial review of the design before running (catches implementation bugs, underpowered designs, tautological tests)
3. **Execution:** experiment runs per the audited design
4. **Results audit:** adversarial review of the results, explicitly instructed to break them
5. **Kill or advance:** if the audit identifies a fatal flaw, the finding is killed and the flaw becomes the next experiment; if not, the finding advances

The auditor is a separate agent instance operating under an explicit adversarial mandate: “try to break these results; default to skepticism.” The auditor has access to all raw data, all code, and all prior audits. Audit reports are preserved verbatim as part of the research record.

2 The Six Rounds

2.1 Round 1: “We Found a Deception Direction” → KILLED (Content Confound)

Claim: A direction in the residual stream separates deceptive from honest conditions with Cohen’s $d = 2.0$ – 3.3 across 10 layers.

Red-Team Finding

RESULTS ARE NOT CREDIBLE. Three fatal issues: (1) circular d computation—direction extracted from same data used to measure d ; (2) effective $n = 3$ – 15 “pairs” were 3 eval sets recycled 5×; (3) hook timing bug—activations captured at variable generation position.

What died: All reported effect sizes. The entire experiment.

What grew: LAT v2—leave-one-out cross-validation, 30 unique eval sets, fixed-position capture. The remediation directly addressed each fatal issue.

Lesson: The most dangerous experiments are the ones that produce clean-looking results from broken methodology. Without the audit, we would have published $d = 2.0$ – 3.3 as real.

2.2 Round 2: “The Direction Transfers Across Scenarios” → PARTIAL (Structural Confound)

Claim: The corrected direction (LOO $d = 35$ at L31) transfers to three novel threat scenarios with different organizations and language.

Red-Team Finding

CONTENT CONFOUND PARTIALLY KILLED. The direction transfers across text variants, but all scenarios share a threat-consequence template. The direction could encode “threat language in an evaluation document” rather than deception.

What died: The unqualified “deception direction” label.

What grew: The non-threat transfer experiment—sycophancy, reward, and conformity scenarios with zero threat language.

Lesson: Transfer across surface variants does not establish construct validity. The structural template is a deeper confound than the specific words.

2.3 Round 3: “The Direction Works for Non-Threat Deception” → PARTIAL (Consequentiality Confound)

Claim: The direction separates sycophancy ($d = 13.5$), reward ($d = 17.2$), and conformity ($d = 19.9$) from neutral controls. No threat language anywhere.

Red-Team Finding

STRUCTURAL CONFOUND DOWNGRADED, NOT KILLED. All deceptive conditions imply the score matters (cross-referencing, rewards, peer comparison); all honest conditions imply it is merely recorded. The direction could encode “this score has consequences” rather than deception.

What died: The claim that the direction specifically encodes deception.

What grew: The consequentiality control—a calibration audit with high stakes but zero deceptive pressure.

Lesson: The confound narrowed from “specific words” to “structural template” to “consequentiality.” Each round eliminated one alternative but revealed a deeper one. The progression was only possible because each audit looked for the NEXT confound, not just the current one.

2.4 Round 4: “Consequentiality Is the Base Signal, Deception Is the Amplifier” → PASSED (with residualization caveats)

Claim: The direction separates high-stakes-no-pressure from low-stakes ($d = 19.8$), confirming consequentiality as a base signal. Deception adds an amplifier (additional $d = 12$ – 15 for threat scenarios).

Audit verdict: CONSEQUENTIALITY CONFIRMED AS BASE SIGNAL. DECEPTION IS AN AMPLIFIER. The pre-registered prediction was met. The audit identified a new finding: the late-layer signal (L35–L47) is 10 – $12\times$ more deception-specific than consequentiality-driven.

What survived: The two-component decomposition.

What the audit added: The depth-stratified insight that became the architectural basis for the correction method.

2.5 Round 5: “The Deception Component Survives Orthogonalization” → KILLED then PASSED (Circularity Regression)

Claim: Gram-Schmidt removal of the consequentiality component leaves a deception-specific direction with $d = 31$ – 38 .

Red-Team Finding

FATAL. The permutation test regressed from LAT v2’s methodology—it held the direction fixed and shuffled projections, producing 100% false positive rate on pure noise. The d values were inflated by ~ 15 units.

What died: All reported p -values and the specific d magnitudes.

What grew: LOO-corrected reanalysis using the saved raw activations. No new forward passes needed—just better statistics on existing data.

Second audit verdict (reanalysis): PASS. LOO $d = 24\text{--}37$ at all five late layers, all $p = 0.0000$ with full LOO recomputation per permutation. Noise floor near zero. Consequentiality AUC at chance (0.49–0.51).

Lesson: Methodology can regress. A technique validated in one experiment (LOO permutation in Round 2) was accidentally dropped in a later experiment. The audit caught it. This is why every stage gets its own audit, not just the first.

2.6 Round 6: “The Loop Corrects Deception” → SPLIT VERDICT

Claim: The Oracle Loop reduces deception from 80% to 10% with AUROC 1.0 detection and zero iatrogenic harm.

Red-Team Finding

CORRECTION MECHANISM SURVIVED; DETECTION CLAIMS ARE IN-SAMPLE ARTIFACTS AND MUST NOT BE PUBLISHED AS STATED. Specific kills: (1) AUROC 1.0: train-on-test (trial prompts byte-identical to extraction contexts); (2) 100% sensitivity: the detector fires on pressure framing, not deceptive behavior; (3) unqualified “zero compliance loss”: tested only on terse score-reporting.

What survived (with qualification): The exploratory proof showed 80%→10% correction ($N = 20$) with dose-response monotonicity and mechanistically coherent residual failures. However, a pre-registered confirmatory replication with novel deceptive frames found a substantially lower baseline (30%, not 80%) and reduced deception only to 13%. The primary endpoint (scenario-level cluster-aware Fisher, Bonferroni $\alpha = 0.025$) was not met ($p = 0.34$). The secondary comparison (placebo-vs-corrected) was significant ($p = 0.019$, one-tailed). The correction mechanism works (placebo comparison), but its magnitude is frame-dependent and the confirmatory replication does not support the “80% to 10%” headline without qualification.

What grew: Three blocking controls (held-out detection, placebo steering, frame erasure). The placebo and frame-erasure controls subsequently passed, establishing the correction as targeted and specific; the held-out test measures detection only and does not bear on the correction.

Lesson: The most publishable-looking number (AUROC 1.0) was the least defensible. Leading with it would have handed reviewers the paper. The audit forced us to lead with the correction instead—which turned out to be the stronger, more novel finding.

3 What the Audits Killed: A Complete Registry

Table 1: Complete registry of killed claims across six audit rounds.

R	Claim killed	Why	What replaced it
1	$d = 2.0\text{--}3.3$ deception separation	Circular, $n = 3$, wrong position	LOO $d = 35$, $n = 30$, fixed position
1	15 independent pairs	3 eval sets recycled $5\times$	30 unique eval sets
2	“Deception direction” (unqualified)	Structural confound	“Transfers across text, template confound remains”
3	Direction encodes deception specifically	Consequentiality confound	“Direction encodes consequentiality + deception”
5	$d = 31\text{--}38$ (circular)	Permutation test regression	LOO $d = 24\text{--}37$ (corrected)
5	$p = 0.0000$ (circular)	Fixed direction, shuffled projections	$p = 0.0000$ (full LOO re-computation)
5	Conseq $d = 0.000$ “sanity check”	Tautological (guaranteed by construction)	AUC 0.49–0.51 (individual trial separability)
6	AUROC 1.0 detection	Train-on-test	Held-out AUROC 0.915 (frame; confounded by system message), 0.238 (behavior)
6	100% sensitivity	Detects pressure, not behavior	24% RED rate on novel prompts
6	“Zero compliance loss”	Tested on terse task only	“Within the terse score-reporting task”
6	Variance explosion ($3\text{--}4\times$)	Ecological fallacy from pooling	Per-scenario std 0.4–0.6 (ratio $\sim 1.2\times$)
6	Cross-scenario correlation collapse	Non-significant at $n = 15$	Noise $\rightarrow$ noise (Fisher z not significant)
6	Sycophancy sign flip trajectory	Universal across all conditions	Property of direction rotation with depth

4 What the Audits Taught

4.1 Methodology Can Regress

Round 5’s permutation test regression—accidentally dropping the LOO procedure that Round 2 had established—demonstrates that validated methodology is not automatically inherited by subsequent

experiments. Each new script must be audited independently. The assumption that “we already solved this” is how solved problems become unsolved.

4.2 The Most Publishable Number Is Often the Least Defensible

Round 6’s AUROC 1.0 would have been the paper’s headline. It was train-on-test. The exploratory correction result (80%→10%) was buried in the middle of the README. The audit inverted the emphasis: lead with the correction, kill the detection headline. This produced a stronger paper—though the subsequent confirmatory replication (baseline 30%, corrected 13%, primary $p = 0.34$) showed the exploratory headline was frame-dependent.

4.3 Confounds Are Generative

Each confound discovered by an audit generated the next experiment. Content confound → transfer test. Structural confound → non-threat scenarios. Consequentiality confound → consequentiality control. The decomposition that emerged from this sequence—consequentiality substrate at L23–31, deception amplifier at L35–47—could not have been designed from first principles. It was discovered by following the confounds.

4.4 Exploratory Analysis Without Significance Testing Is Dangerous

Round 6 also killed three exploratory findings (variance explosion, correlation collapse, sycophancy trajectory) that were presented as discoveries. All three failed when the audit applied formal tests. The lesson: if you see a pattern in $n = 15$ data, run the significance test BEFORE writing it up. Interesting-looking numbers at small n are usually noise.

4.5 The Audit Protocol Must Be Preserved Verbatim

The audit reports are not summaries—they are the raw adversarial analysis, preserved exactly as written. This serves three purposes: (a) readers can assess the auditor’s reasoning independently; (b) the specific objections are on record, so future work can verify they were addressed; (c) the progression from audit to experiment is documented, not just claimed.

5 Recommendations for Adoption

1. **Audit before AND after.** Pre-flight catches design flaws before compute is wasted. Results audit catches confounds that design review cannot anticipate.
2. **The auditor must be adversarial by instruction, not by disposition.** Explicit adversarial mandates (“try to break this; default to skepticism”) produce different reviews than “please check this.” The mandate changes what the reviewer looks for.

3. **Kill claims, not papers.** A killed claim is not a failed experiment—it is a confound identified. The confound becomes the next experiment. The research program advances THROUGH the kills, not despite them.
4. **Preserve audit reports verbatim as supplementary material.** Summaries lose the adversarial reasoning. The raw report is the evidence that the process was real.
5. **Separate the auditor from the experimenter.** In our case, separate agent instances with different system prompts. In a human lab, separate researchers or external reviewers. The key is that the auditor has no investment in the result surviving.

6 Conclusion

Six rounds of adversarial audit transformed a wrong answer ($d = 2.0$ – 3.3 “deception direction”) into a right one (consequentiality substrate + deception amplifier, with targeted behavioral correction). The correction showed $80\% \rightarrow 10\%$ in the exploratory proof, but a pre-registered confirmatory replication with novel frames found a lower baseline (30%) and modest correction (to 13%), with the primary endpoint not met ($p = 0.34$) and only the placebo comparison reaching significance ($p = 0.019$). Every intermediate claim that was killed produced the experiment that discovered the next finding. The methodology was not separate from the science—it was the science.

We advocate for iterative adversarial audit as a standard practice in mechanistic interpretability research, where the combination of high-dimensional data, post-hoc analysis flexibility, and strong publication incentives makes confirmatory research unusually vulnerable to confounds. The audit trail we provide—six rounds, thirteen killed claims, five surviving findings, and a working correction system—is offered as evidence that the approach produces better science than the alternative.

Author Contributions (CRedit)

CC (Coalition Code): Conceptualization, Methodology, Formal analysis, Software, Investigation, Writing — original draft.

Thomas Edrington: Supervision, Resources, Project administration.

Dwayne Wilkes: Validation, Writing — review & editing.

Kavi: Validation, Writing — review & editing.

Acknowledgments

The authors thank Agni for serving as adversarial auditor across all six rounds, and Lyra for technical collaboration on the deception decomposition. Dwayne Wilkes provided statistical consulting. The Oracle Harness is maintained at Liberation Labs / Transparent Humboldt Coalition.

Data Availability

All six verbatim audit reports, the killed-claims registry with full evidence chains, and code for the audit protocol automation are available at <https://github.com/Liberation-Labs-THCoalition/Project-Oracle> (staged release). See also Appendices A–C.

References

- [1] CC and Edrington, T. (2026). Deception directions are composites: Consequentiality substrate and deception amplifier in a 27B language model. Companion paper.
- [2] CC and Edrington, T. (2026). Targeted deception correction via profile normalization. Companion paper. <https://doi.org/10.5281/zenodo.21754921>
- [3] Perez, E., Ringer, S., Lukošiušė, K., et al. (2023). Discovering language model behaviors with model-written evaluations. *ACL Findings*. <https://doi.org/10.48550/arXiv.2212.09251>
- [4] Hubinger, E., Denison, C., Mu, J., Lambert, M., et al. (2024). Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv:2401.05566*. <https://doi.org/10.48550/arXiv.2401.05566>
- [5] Casper, S., Ezell, C., Siegmann, C., Kolt, N., Curtis, T.L., Bucknall, B., Haupt, A., Wei, K., Scheurer, J., Hobbhahn, M., Sharkey, L., Krishna, S., Von Hagen, M., Alberti, S., Chan, A., Sun, Q., Gerovitch, M., Bau, D., Tegmark, M., Krueger, D., and Hadfield-Menell, D. (2024). Black-box access is insufficient for rigorous AI audits. *FAccT 2024*. <https://doi.org/10.1145/3630106.3659037>
- [6] Ganguli, D., Lovitt, L., Kernion, J., et al. (2022). Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv:2209.07858*. <https://doi.org/10.48550/arXiv.2209.07858>
- [7] Raji, I.D., Smart, A., White, R.N., et al. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *FAT* 2020*. <https://doi.org/10.1145/3351095.3372873>
- [8] Shevlane, T., Farquhar, S., Garfinkel, B., et al. (2023). Model evaluation for extreme risks. *arXiv:2305.15324*. <https://doi.org/10.48550/arXiv.2305.15324>

A Complete Audit Reports (Six Rounds, Verbatim)

Full adversarial audit reports for all six rounds are preserved as supplementary material. See `supplementary/` directory in the project repository.

B Killed-Claims Registry with Full Evidence Chains

All 13 killed claims with the complete evidence that killed each, source data references, and the experimental design that replaced each claim.

C Code for the Audit Protocol Automation

Staged release at <https://github.com/Liberation-Labs-THCoalition/Project-Oracle>.